

Create a Rotating 3D View of Saturn's Rings from Cassini Data in Google Colab

Evan Shipley-Friedt | ENGL 393 | August 4, 2026

Part 1: Audience analysis

These instructions are written for beginning undergraduate planetary-science research assistants in the University of Maryland Department of Astronomy. The department studies astronomical systems through observation and computation. Its undergraduates can work with faculty or graduate students on focused research questions (University of Maryland Department of Astronomy, n.d.-a). A new assistant may understand the physical meaning of a measurement but still need a reliable way to inspect an unfamiliar data product. The audience values scientific accuracy. They also need a result that can be reproduced during a research-group conversation.

I am writing as a UMD undergraduate physics student with coursework that has used Python and MATLAB. The Physics major expects students to analyze quantitative results and interpret scientific data (University of Maryland, n.d.). My role here is to guide a neighboring undergraduate audience through a limited computational task, not to claim expertise as an Astronomy Department employee. The final GIF gives the reader a quick view of radial ring structure while making the model's limits visible.

Part 2: Technical instructions

Output files

A complete run creates:

- `cassini_optical_depth.png`
- `saturn_rings_3d.png`
- `saturn_rings_rotation.gif`

Introduction

Cassini's radio-science experiments measured how a radio signal changed while passing through Saturn's rings. The resulting occultation profile gives a ring radius and a normal optical-depth measurement at each recorded position (National Aeronautics and Space Administration, n.d.-a). A high optical depth means less of the signal passed through that part of the rings. In this procedure, that measurement controls how many representative points appear at a radius.

The result is a data-based display rather than a particle simulation. Its points help reveal radial structure, but they are not individually observed particles. The camera rotates around a still model so the finished GIF can show the depth of the scene without claiming to reproduce ring motion.

Scientific and technical background

Saturn's rings contain many narrow features instead of one uniform sheet (National Aeronautics and Space Administration, n.d.-b). The selected PDS product comes from Cassini's May 3, 2005 X-band egress occultation. Its radial resolution is 10 kilometers. The table contains 28,888 measurements.

The first table column contains radius in kilometers. The seventh column contains normal optical depth. The graph retains all nonnegative measured values. A separate copy is limited to 3.0 only when the notebook decides how many points to draw. This keeps a few large measurements from producing an excessive number of points without hiding them from the graph.

Warnings and cautions

Scientific limitation: The ring points are visual representatives of the measured radial profile. They are not individual particles observed by Cassini.

Model limitation: The notebook does not calculate gravity or collisions. Rotating the camera does not represent physical ring motion.

Data caution: Keep the PDS filename unchanged. The column positions in these instructions apply to this specific TAU product.

Software requirements

This procedure uses [Google Colab](#), which supplies the required Python environment without a local installation.

Recommended: Signing in to a Google account allows Colab to save the notebook to Google Drive.

Local alternative: The same notebook can run in [JupyterLab](#) or the [VS Code notebook editor](#). Keep the `.ipynb` extension for notebook cells. A converted single-file script uses `.py`. The local environment needs Python 3. Install NumPy and Matplotlib. Pillow saves the animation; IPython or Jupyter provides notebook display support.

Procedure

1. Open and name the notebook

Open [Google Colab](#). Signing in is optional, but it allows automatic saving to Google Drive. Select **New notebook**, then rename the file `saturn_rings_colab.ipynb`.

Check the result The new filename should appear in the upper-left corner.

2. Import the tools and create an output folder

Paste this code into the first cell:

```
# Load the libraries used to download data and build the three visuals.
from pathlib import Path
from urllib.request import Request, urlopen

import matplotlib.pyplot as plt
import numpy as np
from matplotlib.animation import FuncAnimation, PillowWriter
from IPython.display import Image, display

# Keep every image created by the notebook in one folder.
output_folder = Path("visuals")
output_folder.mkdir(exist_ok=True)
```

Run the cell with **Shift+Enter**. NumPy handles the measurements. Matplotlib creates the graph and 3D view. Pillow writes the animation file.

Check the result The cell should finish without red error text. No image appears yet.

3. Download the Cassini profile from NASA PDS

Add a new cell and paste:

```
# Identify the exact Cassini profile and its local filename.
profile_url = (
    "https://pds-rings.seti.org/holdings/volumes/CORSS_8xxx/CORSS_8001/"
    "data/Rev007/Rev007E/Rev007E_RSS_2005_123_X43_E/"
    "RSS_2005_123_X43_E_TAU_10KM.TAB"
)
profile_file = Path("RSS_2005_123_X43_E_TAU_10KM.TAB")

# Request the file and save the returned bytes with the original name.
profile_request = Request(
    profile_url,
    headers={"User-Agent": "Mozilla/5.0"}
)
with urlopen(profile_request) as response:
    profile_file.write_bytes(response.read())

print(f"Downloaded {profile_file.name}")
```

Run the cell. This address points to a documented profile in NASA PDS dataset C0-SR-RSS-4/5-OCC-V2.0 (National Aeronautics and Space Administration, n.d.-a).

The filename describes the product. TAU identifies the reconstructed optical-depth profile; it is not the file extension. The .TAB extension means this file is the ASCII table that contains the measurements. A similarly named .LBL file is the detached label that describes the table, so it should not be loaded as numeric data.

Check the result Colab should print Downloaded RSS_2005_123_X43_E_TAU_10KM.TAB.

4. Load the ASCII table and confirm its shape

Add and run:

```
# Read the comma-separated measurements into a two-dimensional NumPy array.
profile_data = np.loadtxt(profile_file, delimiter=",")

# Display the table dimensions before selecting any columns.
print(f"Rows: {profile_data.shape[0]}")
print(f"Columns: {profile_data.shape[1]}")
```

NumPy's `loadtxt` changes the consistently formatted text rows into a numerical array (NumPy Developers, n.d.). The row count confirms that the entire data product loaded. The column count confirms that later column positions will refer to the expected measurements.

Check the result Colab should report 28,888 rows and 13 columns.

5. Create the 3D figure and Saturn's surface

Add and run:

```
# Create a three-dimensional plotting area.
ring_figure = plt.figure(figsize=(7.2, 7.2))
ring_axis = ring_figure.add_subplot(111, projection="3d")

# Build a regular grid of longitude and polar angles for Saturn only.
theta = np.linspace(0.0, 2.0 * np.pi, 64)
phi = np.linspace(0.0, np.pi, 32)
theta_grid, phi_grid = np.meshgrid(theta, phi)

# Convert the angular grid into three surface-coordinate arrays.
saturn_polar_scale = 0.90
saturn_x = np.sin(phi_grid) * np.cos(theta_grid)
saturn_y = np.sin(phi_grid) * np.sin(theta_grid)
saturn_z = saturn_polar_scale * np.cos(phi_grid)

saturn_surface = ring_axis.plot_surface(
    saturn_x,
    saturn_y,
    saturn_z,
    color="#d7b878",
    edgecolor="none"
)
```

The `meshgrid` supplies every pairing of the two surface angles. The 64 longitude values and 32 polar values create a surface that looks smooth without adding unnecessary rendering time. Saturn is wider around its equator than from pole to pole. The `0.90` polar scale approximates the ratio between those published dimensions (National Aeronautics and Space Administration, n.d.-c).

6. Select and prepare the measured columns

Add and run:

```
# Select radius from column 1 and optical depth from column 7.
radius_km = profile_data[:, 0]
optical_depth = profile_data[:, 6]

# Remove incomplete rows and change small negative noise values to zero.
valid_measurement = np.isfinite(radius_km) & np.isfinite(optical_depth)
radius_km = radius_km[valid_measurement]
optical_depth = np.maximum(optical_depth[valid_measurement], 0.0)

# Cap optical depth at 3.0 only for the marker-count calculation.
display_depth = np.minimum(optical_depth, 3.0)

print(f"Radius range: {radius_km.min():.1f} to {radius_km.max():.1f} km")
```

Python begins counting at zero, so index 6 selects the seventh column. `optical_depth` remains unchanged for the measured profile graph. `display_depth` is the capped copy used to set marker density. Values above 3.0 receive the same maximum marker count, which keeps isolated peaks from overwhelming the 3D view.

Check the result The radius should begin near 72,770 km and end near 144,988 km.

7. Plot the optical-depth profile

Add and run:

```
# Create a line graph of measured optical depth at each radius.
profile_figure, profile_axis = plt.subplots(
    figsize=(9, 4.8), constrained_layout=True
)
profile_axis.plot(
    radius_km / 1000.0,
    optical_depth,
    color="#d99a39",
    linewidth=0.8
)
profile_axis.set(
    title="Cassini RSS Saturn Ring Profile",
    xlabel="Distance from Saturn's center (thousand km)",
    ylabel="Normal optical depth"
)
profile_axis.grid(alpha=0.25)

# Save the graph and close its plotting window.
profile_image = output_folder / "cassini_optical_depth.png"
profile_figure.savefig(profile_image, dpi=180)
plt.close(profile_figure)

# Open the saved graph beneath the cell.
display(Image(filename=str(profile_image)))
```

The graph confirms that the correct columns loaded before the profile becomes a 3D display. It also makes the relationship between radius and optical depth easier to see.

Check the result A detailed line with many narrow peaks should appear below the cell.

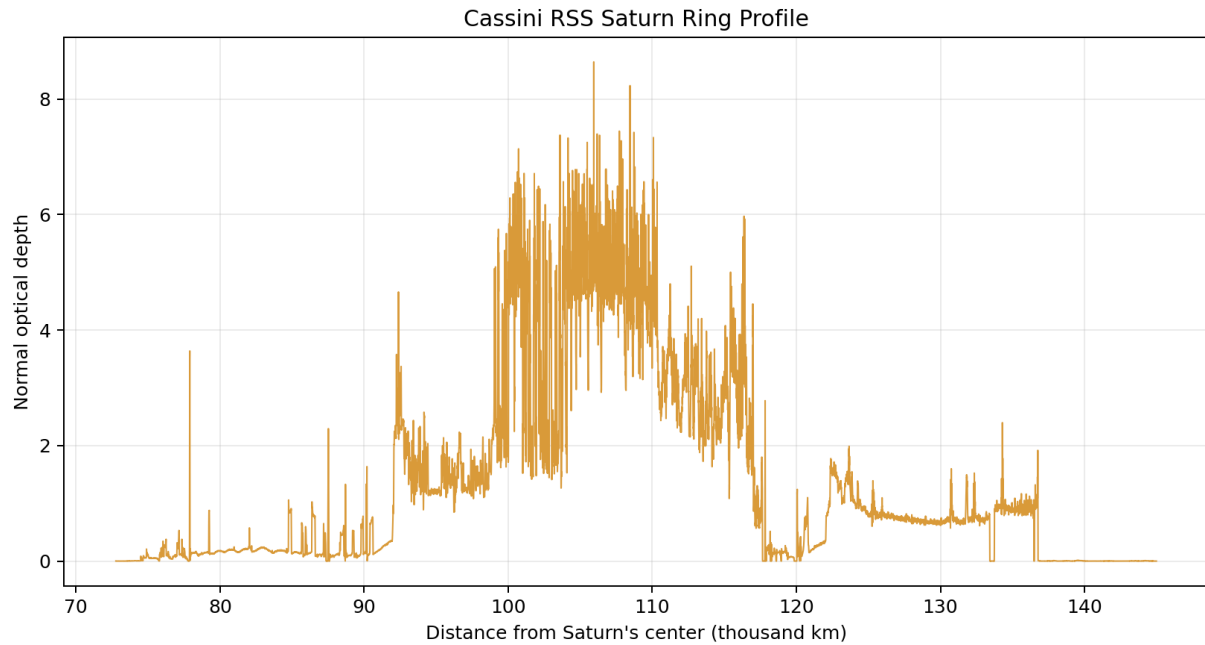


Figure 1. Moving from left to right means moving farther from Saturn's center. High peaks mark ring regions that blocked more of Cassini's radio signal, while low areas show more transparent gaps. The next step represents those peaks with a higher concentration of points; the graph does not measure ring mass.

8. Convert the profile into representative ring points

Add and run:

```
# Sample about 1,500 radii so the animation remains quick to render.
saturn_radius_km = 60268.0
profile_stride = max(1, len(radius_km) // 1500)
sample_radius_km = radius_km[::profile_stride]
sample_depth = display_depth[::profile_stride]

# Store the horizontal coordinates as simple lists while the loop creates them.
ring_x = []
ring_y = []
random_generator = np.random.default_rng(7)
points_per_depth_unit = 2

for radius_value, depth_value in zip(sample_radius_km, sample_depth):
    # Give every sampled radius one base marker, then add two per depth unit.
    number_of_points = 1 + int(
        depth_value * points_per_depth_unit
    )

    # Spread each sampled radius around a complete circle.
    point_angle = random_generator.uniform(
        0.0, 2.0 * np.pi, number_of_points
    )
    display_radius = radius_value / saturn_radius_km
    ring_x.extend(display_radius * np.cos(point_angle))
    ring_y.extend(display_radius * np.sin(point_angle))

# Convert the finished horizontal coordinates into arrays for Matplotlib.
ring_x = np.array(ring_x)
ring_y = np.array(ring_y)

# The source profile has no vertical measurements, so the rings remain flat.
ring_z = np.zeros(ring_x.size)

print(f"Representative points: {ring_x.size:,}")
```

Dividing by Saturn's published radius places each Cassini radius relative to the planet's size. The stride keeps about 1,500 regularly spaced measurements and shortens animation time without losing narrow radial changes.

Every sampled radius receives one base marker so transparent regions remain as sparse gaps. `points_per_depth_unit = 2` adds two markers per full optical-depth unit. A zero-depth measurement creates one marker. At the 3.0 display limit, six are added for a total of seven.

Random angles spread points around each circle instead of stacking the profile on one line. Seed 7 repeats those placements after a clean run; it does not generate the Cassini data. The flat z array avoids inventing thickness. Saturn's main rings are only about 10 meters thick, which would appear flat at this scale (National Aeronautics and Space Administration, n.d.-b).

Check the result Colab should print Representative points: 4,353.

9. Add the rings and save a progress view

Add and run:

```
# Add every ring coordinate to the same 3D axes as Saturn.
ring_points = ring_axis.scatter(
    ring_x,
    ring_y,
    ring_z,
    color="#c9a56b",
    s=1.5,
    alpha=0.68,
    linewidths=0
)

# Set the camera direction used by the still progress image.
view_elevation = 28.0
view_azimuth = 35.0
ring_axis.view_init(elev=view_elevation, azimuth=view_azimuth)

# Save and open the progress image beneath the cell.
preview_image = output_folder / "saturn_rings_3d.png"
ring_figure.savefig(preview_image, dpi=180)
display(Image(filename=str(preview_image)))
```

The image confirms that the radial measurements now form a flat ring plane around Saturn. At this stage, Matplotlib may draw some far-side ring points over the planet. Step 11 corrects that display problem.

Check the result Saturn should appear inside a flat ring plane. Ring points may still appear in front of the planet.

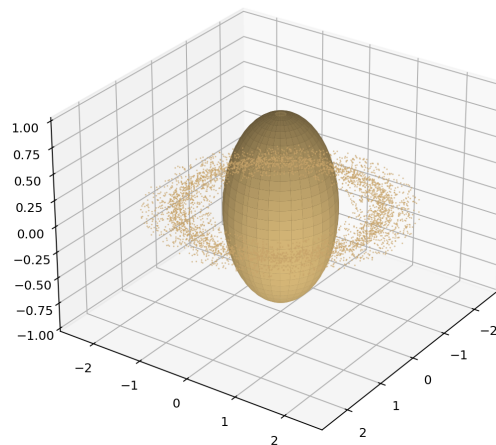


Figure 2. The progress view shows the measured ring profile around Saturn before the visibility correction. Some far-side points may appear over the planet at this stage.

10. Finish the 3D appearance

Add and run:

```
# Apply the finished colors to the same live figure.
ring_figure.set_facecolor("#05070b")
ring_axis.set_facecolor("#05070b")
saturn_surface.set_facecolor("#d7b878")
saturn_surface.set_edgecolor("none")

# Fit the axes around the largest measured radius.
ring_limit = radius_km.max() / saturn_radius_km + 0.18
ring_axis.set(
    xlim=(-ring_limit, ring_limit),
    ylim=(-ring_limit, ring_limit)
)
ring_axis.set_zlim(-ring_limit * 0.72, ring_limit * 0.72)
ring_axis.set_box_aspect((1, 1, 0.62))

# Remove the axis lines and add the finished title.
ring_axis.set_axis_off()
ring_axis.set_title(
    "Saturn's Rings from a Cassini Radio-Occultation Profile\n"
    "Point concentration represents measured optical depth",
    color="white",
    pad=12,
    fontsize=11
)
```

These settings change the presentation without changing the Cassini measurements or ring coordinates. The axis limits use the largest measured radius so the full profile remains visible.

11. Hide ring points behind Saturn

Behind the scenes: correct the 3D view Matplotlib orders supported 3D objects by their average position along the viewing direction. That approximation can place a distant ring marker over Saturn as the camera rotates (Matplotlib Development Team, n.d.-a). This helper checks whether Saturn lies between each marker and the camera.

```
def visible_ring_mask(elevation, azimuth):
    # Convert the viewing angles into a direction from the scene to the camera.
    elevation = np.deg2rad(elevation)
    azimuth = np.deg2rad(azimuth)
    camera_direction = np.array([
        np.cos(elevation) * np.cos(azimuth),
        np.cos(elevation) * np.sin(azimuth),
        np.sin(elevation)
    ])

    # Rescale the flattened planet so the visibility test can use a sphere.
    saturn_shape = np.array([1.0, 1.0, saturn_polar_scale])
    scaled_points = np.column_stack(
        (ring_x, ring_y, ring_z)
    ) / saturn_shape
    scaled_camera = camera_direction / saturn_shape
    scaled_camera = scaled_camera / np.linalg.norm(scaled_camera)

    # Find the position on each viewing line nearest Saturn's center.
    distance_to_closest_point = -(scaled_points @ scaled_camera)
    closest_points = (
        scaled_points
        + distance_to_closest_point[:, None] * scaled_camera
    )

    # Saturn blocks a marker when that nearest position lies inside the sphere.
    blocked = (
        (distance_to_closest_point > 0.0)
        & (np.sum(closest_points**2, axis=1) <= 1.0)
    )
    return ~blocked

# Apply the correction to the initial camera angle.
visible_point = visible_ring_mask(view_elevation, view_azimuth)
ring_points._offsets3d = (
    ring_x[visible_point],
    ring_y[visible_point],
    ring_z[visible_point]
)
```

Dividing by `saturn_shape` temporarily turns the flattened planet into a unit sphere. For each ring marker, `closest_points` contains the position along the viewing line that comes nearest Saturn's center. The marker is hidden only when that position is in front of it and falls inside the sphere. This is a simplified form of the standard ray-sphere intersection method used in computer graphics (Pharr et al., 2023).

12. Rotate the camera and save the GIF

Add and run:

```
# Thirty frames at ten frames per second create a three-second loop.
# Lowering the frame count speeds up rendering but makes larger camera jumps.
gif_frame_count = 30

def rotate_view(frame):
    viewing_angle = frame * 360.0 / gif_frame_count

    # Recalculate which ring markers Saturn blocks at this camera angle.
    visible_point = visible_ring_mask(view_elevation, viewing_angle)
    ring_points._offsets3d = (
        ring_x[visible_point],
        ring_y[visible_point],
        ring_z[visible_point]
    )

    ring_axis.view_init(elev=view_elevation, azim=viewing_angle)
    return (ring_points,)

# Build the frames and save them as one GIF.
rotation = FuncAnimation(
    ring_figure,
    rotate_view,
    frames=gif_frame_count,
    interval=100
)
gif_file = output_folder / "saturn_rings_rotation.gif"
rotation.save(gif_file, writer=PillowWriter(fps=10), dpi=105)
plt.close(ring_figure)

print(f"Saved {gif_file}")
display(Image(filename=str(gif_file)))
```

FuncAnimation calls rotate_view for every frame. The visibility correction keeps the far side of the rings behind Saturn at each angle. PillowWriter combines the frames into a GIF (Matplotlib Development Team, n.d.-b). The 105 dpi setting limits image size and file size; it does not change the data.

Check the result Colab should print Saved visuals/saturn_rings_rotation.gif, then display one smooth rotation.

gif_frame_count can be lowered to shorten rendering time. The camera will turn farther between frames without changing the ring data.

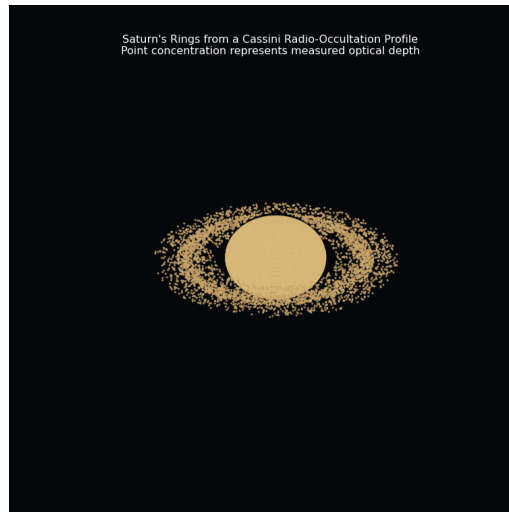


Figure 3. The camera rotates around a fixed display of the Cassini profile. The animation reveals the 3D viewing geometry without representing the physical motion of Saturn's rings.

To confirm that the completed notebook works from a clean state, select **Runtime > Restart session and run all**. The graph should appear before the progress 3D view. The GIF should appear last without red error text.

Part 3: References

Google. (n.d.). *Google Colab*. <https://colab.research.google.com/>

Last, S. (2019). *Writing instructions*. In *Technical writing essentials*. BCcampus. <https://pressbooks.bccampus.ca/technicalwriting/chapter/writinginstructions/>

Matplotlib Development Team. (n.d.-a). *mpl_toolkits.mplot3d.axes3d.Axes3D*. <https://matplotlib.org/stable/api/toolkits/mplot3d/axes3d.html>

Matplotlib Development Team. (n.d.-b). *Animation API*. https://matplotlib.org/stable/api/animation_api.html

Microsoft. (n.d.). *Jupyter notebooks in Visual Studio Code*. <https://code.visualstudio.com/docs/datascience/jupyter-notebooks>

National Aeronautics and Space Administration. (n.d.-a). *Cassini RSS ring occultation profiles V2.0*. Planetary Data System. <https://pds.nasa.gov/ds-view/pds/viewDataset.jsp?dsid=CO-SR-RSS-4%2F5-OCC-V2.0>

National Aeronautics and Space Administration. (n.d.-b). *Rings. Cassini*. <https://science.nasa.gov/mission/cassini/science/rings/>

National Aeronautics and Space Administration. (n.d.-c). *Cassini: Quick facts*. <https://science.nasa.gov/wp-content/uploads/2023/09/cassini.pdf>

NumPy Developers. (n.d.). *numpy.loadtxt*. <https://numpy.org/doc/stable/reference/generated/numpy.loadtxt.html>

Pharr, M., Jakob, W., & Humphreys, G. (2023). *Spheres*. In *Physically based rendering: From theory to implementation* (4th ed.). <https://www.pbr-book.org/4ed/Shapes/Spheres>

Project Jupyter. (n.d.). *Installing Jupyter*. <https://jupyter.org/install>

TechSmith. (n.d.). *How to create step-by-step instructions using visuals*. <https://www.techsmith.com/blog/create-instructions-using-visuals/>

University of Maryland. (n.d.). *Physics major. Undergraduate Catalog*. <https://academiccatalog.umd.edu/undergraduate/colleges-schools/computer-mathematical-natural-sciences/physics/physics-major/>

University of Maryland Department of Astronomy. (n.d.-a). *Astronomy major*. <https://www.astro.umd.edu/education/undergraduate/astronomy-major>

University of Maryland Department of Astronomy. (n.d.-b). *Undergraduate research*. <https://www.astro.umd.edu/education/undergraduate/undergraduate-research>